

Lab 4.2: KiCad Schematic & PCB

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** Capturing the Lab 4-1 I/O circuit as a KiCad schematic and routing it into a real *Arduino Uno shield* – a two-layer PCB that stacks directly onto the Uno’s own header pins – optionally fabricated via PCBWay

Objectives

- Capture the *exact* circuit you breadboarded and tested in **Lab 4-1** as a schematic in **KiCad**, assign footprints, and lay it out into a two-layer PCB that passes Electrical Rules Check (ERC) and Design Rules Check (DRC).
- Design the board in **Arduino Uno shield form factor**: the exact board outline and header positions the Uno itself uses, so your finished PCB *stacks directly* onto the Uno’s own pins – no jumper wires – instead of connecting through a generic pin header.
- Generate Gerber and drill files – the manufacturing handoff format every PCB fab, including PCBWay, expects.
- (Optional) Submit your Gerbers to **PCBWay** and order your own manufactured board.

Quick Reference: The Circuit You’re Capturing

This is the same 18-pin Arduino Uno R3 circuit from Lab 4-1 – nothing new to design, only to re-express as a schematic and land it on the Uno’s real shield headers instead of loose jumper wires.

Port bits	Uno pins	Direction	Shield header	Function
PB0–PB3	D8–D11	Input, pull-up	Digital (8–13)	SW1–SW4 push buttons
PB4, PB5	D12, D13	Output	Digital (8–13)	DIG1, DIG2 digit-select transistor bases
PC0–PC5	A0–A5	Output	Analog In	Segments a,b,c,d,e,f (shared by both digits)
PD2	D2	Output	Digital (0–7)	Segment g (shared by both digits)
PD3–PD7	D3–D7	Output	Digital (0–7)	LED1–LED5, individually addressable
–	GND	–	Power <i>or</i> Digital (8–13)	Common ground return

Every signal lands on one of the Uno’s **four standard shield header strips** – see the box below before you place a single connector symbol.

Quick Reference: Arduino Uno Shield Header Layout

A real Uno shield does not use generic pin headers wherever convenient – it must reproduce the Uno’s own four female header strips, in their exact positions, so the board stacks and every pin lines up:

Header	Pins	Signals (in order)
Digital (8-13)	8	AREF, GND, D13, D12, D11, D10, D9, D8
Digital (0-7)	8	D7, D6, D5, D4, D3, D2, D1(TX), D0(RX)
Power	7	IOREF, RESET, 3V3, 5V, GND, GND, VIN
Analog In	6	A0, A1, A2, A3, A4, A5

The two **Digital** headers run along one long edge of the board (top, in the usual orientation); **Power** and **Analog In** run along the *opposite* edge (bottom). We only wire the pins this circuit actually uses (D2-D13, A0-A5, one GND) – D0/D1 (USB-serial), AREF, and the 3V3/5V/VIN/RESET/IOREF power pins stay unconnected on this board, exactly as they were left unused in Lab 4-1.

The famous gap: the horizontal spacing between the last pin of the Digital (0-7) header and the first pin of the Digital (8-13) header is *not* a clean multiple of the 0.1 in grid every other pin sits on – an original Arduino Uno design quirk that every compatible shield must reproduce *exactly*. Do not eyeball or round this gap: copy the position from an authoritative Arduino Uno shield footprint/outline (KiCad's bundled Arduino connector library, if present, or Arduino's own published Uno R3 dimensional drawing) rather than placing a plain 0.1 in header and hoping it lines up.

Bill of Materials

Qty	Part	Notes
4	Tactile push button (6 mm THT)	SW1-SW4, no external resistor
5	LED, 3 mm or 5 mm	LED1-LED5
5	Resistor, 330 Ω	Current limiting for LED1-LED5
2	7-segment display, common cathode	DIGIT1 (tens), DIGIT2 (units)
7	Resistor, 330 Ω	Shared segment resistors a-g
2	NPN transistor (e.g. 2N3904)	Digit-select low-side switches, Q1/Q2
2	Resistor, 1 k Ω	Transistor base resistors
4	Female header, 2.54 mm (8, 8, 7, and 6 positions)	The Uno's standard shield header set (Digital 8-13, Digital 0-7, Power, Analog In) – <i>not</i> generic pin headers; these must stack directly onto the Uno's own male header pins

KiCad Workflow Overview

1. **Schematic capture (Schematic Editor):** place a symbol for every part in the bill of materials above, wire them exactly as you wired the Lab 4-1 breadboard, and label every port-bit net that connects back to the Uno.
2. **Electrical Rules Check (ERC):** catch unconnected pins, conflicting outputs, and missing power symbols *before* laying out copper.
3. **Footprint assignment:** attach a physical PCB footprint (THT pad pattern) to every symbol.
4. **PCB layout (PCB Editor):** import the netlist, place components, reproduce the Uno's exact shield outline and header positions (not a board outline of your own choosing), and route every trace.
5. **Design Rules Check (DRC):** catch clearance violations, unrouted nets, and footprint overlaps.
6. **Manufacturing output:** plot Gerber files (one per copper/silkscreen/ mask layer) and a drill file – the format every fab house, including PCBWay, accepts.

Quick Reference: Symbols You Need

KiCad symbol	Represents
Device:R	Every 330 Ω and 1 k Ω resistor
Device:LED	LED1-LED5
Device:Q_NPN_BCE	Digit-select transistors Q1, Q2
Switch:SW_Push	SW1-SW4
A 7-segment, common-cathode display symbol (search “7SEG” in the library browser)	DIGIT1, DIGIT2 (confirm <i>common cathode</i> , not anode, before placing)
An Arduino Uno shield connector footprint/symbol (search “Arduino” or “Shield”; use KiCad’s bundled Arduino connector library if your install has one)	The four female header strips from the Shield Header Layout box above – <i>do not</i> substitute generic Connector_Generic pin headers, since the shield’s mechanical fit depends on the real footprint, not just matching hole spacing by eye
power:GND, power:+5V	Power symbols – required by ERC on every sheet that uses power nets

This board is a *peripheral shield*, not a second computer: it carries no microcontroller, crystal, or reset circuit of its own – the Uno you already have supplies all of that, exactly as it did in Lab 4-1.

Example Output (Illustrative Draft)

Figures 1 and 2 show, roughly, the *kind* of board you are aiming for – a rough draft sketch, *not* a prescriptive layout to copy. Your own component placement, routing, and even board outline will (and should) look different once you complete the exercises below; every real design decision – where each part goes, how each trace is routed – is yours to make.

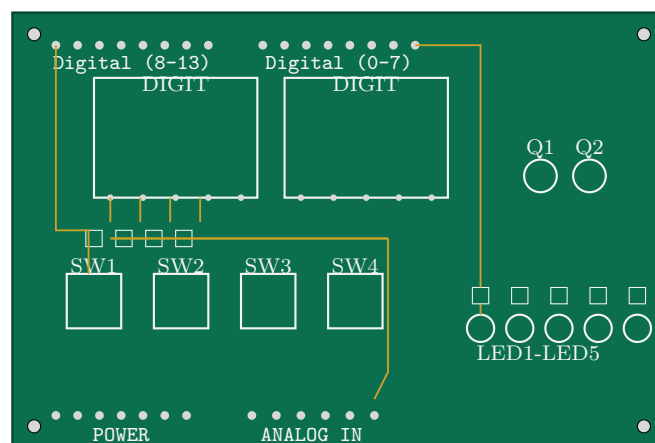


Figure 1: Illustrative top-copper/silkscreen preview – *not to scale, not a routing solution to copy*. The two Digital headers run along the top edge (note the gap – see the Shield Header Layout box above), POWER and ANALOG IN along the bottom edge. Gray pads are the Uno’s own shield header pins; the thin gold lines are illustrative traces only.

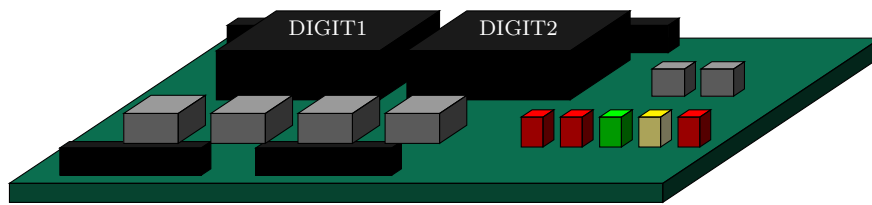


Figure 2: Illustrative 3D-rendered preview of the same draft layout – a simplified stand-in for what KiCad’s own 3D Viewer will show once your footprints have 3D models assigned. The two digital headers run behind the tall displays from this angle (only their edges peek out) – the 2D preview above is the reliable reference for header positions. Component colors and proportions are approximate.

Quick Experiments (Try It)

Try It 1: One symbol, one footprint, one look

Before wiring the whole schematic, place a single `Device:R` symbol, open its **Footprint** field, and assign a THT footprint (e.g. `Resistor_THT:R_Axial_DIN0207_L6.3mm_D2.5mm_P10.16mm_Horizontal`). Switch to the PCB Editor’s footprint preview and confirm you can see the two-pad footprint before you commit to using it across all 12 resistors in your design.

In-Lab Exercises (target: 3 hours)

In-Lab Exercise 1: Schematic capture (50 min)

Create a new KiCad project. Place and wire every symbol from the Quick Reference table above, reproducing your Lab 4-1 breadboard circuit net-for-net: buttons to their Digital(8-13) header pins with no external resistor, LEDs through their resistors to their Digital(0-7) header pins, the shared segment resistor bus feeding both 7-segment symbols in parallel and continuing to the Analog In header, and each digit-select transistor’s collector/base/emitter wired as in Lab 4-1. Annotate all references (so every part gets a unique designator like R1, Q2) and run ERC until it reports zero errors.

In-Lab Exercise 2: Footprints (30 min)

Assign a footprint to every symbol: THT axial resistor footprints for all 12 resistors, a 3 mm or 5 mm THT LED footprint (matching your real LEDs) for LED1-LED5, a T0-92 footprint for Q1/Q2, a THT tactile-switch footprint for SW1-SW4, the matching 7-segment display footprint (check your display’s datasheet for pin pitch and confirm it against your physical part), and the four Arduino shield header footprints (Digital 8-13, Digital 0-7, Power, Analog In) – *not* generic pin headers. Re-run ERC – footprint assignment does not change connectivity, so it should still be clean.

In-Lab Exercise 3: PCB layout and routing (60 min)

Update the PCB from the schematic (imports the netlist as ratsnest lines). Unlike a freestanding board, you do *not* get to choose your own outline or header positions here: reproduce the Uno’s own board outline and the exact positions of all four shield headers (roughly 68.6 mm × 53.3 mm / 2.7 in × 2.1 in overall – copy the real dimensions from an authoritative Arduino shield outline/footprint rather than measuring off a photo). Place the remaining components (displays,

buttons, LEDs, transistors) wherever fits well between the fixed headers – displays and their digit labels easy to read from one edge, buttons reachable from the front. Route every trace (a 0.3-0.4 mm signal trace width is a reasonable default for this current level) until the ratsnest is empty, then run DRC until it reports zero errors.

- **Trace/predict:** before routing, look at your ratsnest and predict which net will be hardest to route without crossing others – is it the shared segment bus (7 nets fanning out to 14 physical pads across 2 displays) or something else? Route it first and see if your prediction held.
- Confirm your Digital(0-7)/Digital(8-13) header gap matches your reference outline *exactly* – do not round it to the nearest 0.1 in grid point (see the Shield Header Layout box’s “famous gap” warning).

In-Lab Exercise 4: Manufacturing outputs (25 min)

Plot Gerber files for every copper, mask, and silkscreen layer your board uses, plus a drill file (Excellon format). Open the resulting Gerbers in KiCad’s Gerber Viewer (or the PCB Editor’s 3D Viewer) and visually confirm: every trace you expect to see is present, the board outline is a single closed shape, and no copper overlaps the board edge. Zip the Gerber and drill files together – this zip is what a fab house expects as a single upload.

Optional: Ordering from PCBWay

Submitting your board for real fabrication is entirely optional and at your own expense (a handful of small boards typically costs a few dollars, plus shipping) – it is not required to complete this lab.

1. Create an account at PCBWay and start a new PCB quote.
2. Upload the Gerber/drill zip from Exercise 4. PCBWay’s online viewer will render your board – compare it against your own KiCad 3D view before ordering; a mismatch usually means a missing or mis-plotted layer.
3. Default specs (2 layers, 1.6 mm thickness, HASL surface finish, standard solder mask color) are more than adequate for this design – there is no need to change anything unless you have a specific reason to.
4. Double-check the quantity and shipping method before paying; lead time (fabrication plus shipping) is typically 1-2 weeks for standard service.

If you choose not to order, submit your KiCad project files (schematic, PCB layout, and the Gerber/drill zip) as your deliverable instead.

AI Collaboration

Try these prompts with an AI tool:

- “What KiCad symbol and footprint should I use for a common-cathode 7-segment display, and how do I confirm the pinout matches my physical part before I order a PCB?”
- “What’s a reasonable trace width and clearance for a hobbyist 2-layer PCB carrying under 100 mA per net, and why doesn’t it need to be wider?”
- “What’s the difference between ERC and DRC in KiCad, and why can a design pass one but fail the other?”
- “What are the exact dimensions and header positions of an Arduino Uno R3 shield, including the gap between the two digital headers? Where does that information come from originally?”

Critical check – verify, don’t just accept: If an AI tool gives you a footprint, symbol, or a shield dimension, do not just place or trust it – open the footprint preview and measure it against your physical part’s datasheet pin pitch (or against an authoritative Arduino dimensional drawing, for the shield outline itself) before you route a single trace. Two 7-

segment displays that look identical in a library browser can use different pin spacing, and a plausible-sounding shield dimension repeated confidently across many sources is not the same as a verified one.

Know the limit: AI tools will confidently give you a “safe” trace width or clearance number without asking what copper weight (1 oz vs. 2 oz) or current your specific net actually carries. Treat any such number as a starting point to verify against your fab’s own capability table (e.g. PCBWay’s standard 1 oz/6 mil specification), not as a final answer.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Manufacturability at a tighter tolerance

A different student’s layout uses 0.15 mm (6 mil) trace width and 0.15 mm (6 mil) clearance throughout, matching PCBWay’s advertised *standard* minimum. Another student’s layout uses 0.5 mm (20 mil) trace width and clearance throughout. Which design is more likely to need an *advanced* (costlier) fabrication tier rather than PCBWay’s standard tier, and why does clearance – not just trace width – matter to that answer?

Find the Bug 1: Rounding the famous gap

A student, in a hurry, placed the Digital(0-7) and Digital(8-13) headers using two plain 8-pin, 0.1 in-pitch connectors, and simply butted them together with a clean 0.2 in gap between the last pin of one and the first pin of the other – reasoning that “it’s a header, it’s all 0.1 in spacing.” DRC reports zero errors, and every schematic net is correctly connected to the correct physical pad. Explain why DRC cannot catch this mistake, and predict exactly what happens when the student tries to stack the fabricated shield onto a real Arduino Uno.

Write Code on Paper 1: Hand-write a netlist

On paper, without opening KiCad, write out the netlist for the DIG1 digit-select sub-circuit (Q1, its base resistor, and everything each of Q1’s three terminals connects to) in the simple text format `net_name: pin, pin, pin` – one line per net. You should end up with exactly as many nets as there are electrically distinct connection points in that sub-circuit; state how many that is before you start counting pins.

Draw on Paper 1: Sketch your board before opening KiCad

On paper, sketch your shield’s component placement – both displays, all four buttons, all five LEDs, and both transistors – *before* you open the PCB Editor. Unlike Exercise 3’s board outline and header positions (which are fixed by the Uno’s own dimensions, not yours to choose), component placement is still your decision: draw the four shield headers at their real relative positions first (as a fixed frame you must design around), then sketch where everything else goes inside that frame. Then build your actual layout in Exercise 3 and compare: did your paper placement survive contact with real routing constraints, or did something need to move? What forced the change?

Reflection

In one short paragraph, compare this lab's PCB to Lab 4-1's breadboard. They implement the *exact same* schematic – same parts, same nets, same firmware. What changed between them, and what stayed identical? Consider specifically: which mistakes (a loose jumper wire, a swapped segment pin) are trivial to fix on a breadboard but expensive or impossible to fix once a PCB is fabricated – and what does that imply about how much verification (ERC, DRC, comparing footprints against real datasheets) is worth doing *before* sending Gerbers to a fab house, versus how much verification Lab 4-1's breadboard build needed before you trusted it?

Appendix A: KiCad and PCBWay Troubleshooting

Message / symptom	Likely cause and fix
ERC reports “Pin not connected” on a symbol you know is wired	A wire crossing another wire without a junction dot is <i>not</i> electrically connected in KiCad – add an explicit junction, or move the wire so it meets at a clean T or corner.
ERC reports “conflicting power pin” on your shield header nets	The shield header pins need a net label or hierarchical label matching each pin name (e.g. D8), not a floating power symbol – power symbols are only for true +5V/GND rails.
Footprint preview shows the wrong pad spacing versus your physical part	You picked a footprint by name similarity, not by measuring your part. Measure pin pitch with calipers (or read the datasheet) and re-select a footprint that matches exactly – this is the single most common reason a bring-up board doesn’t fit its parts. A 3 mm LED footprint on a board populated with real 5 mm LEDs is a classic example: ERC never catches it (footprints carry no electrical meaning to ERC), and the part simply won’t seat in undersized pads once the board is populated.
DRC reports “clearance violation” between two traces that look far apart on screen	Zoom in – it is almost always a via annular ring or a trace corner, not the visible trace body. Increase zoom before assuming DRC is wrong.
DRC reports unrouted nets after you believe everything is routed	Check for a net with only <i>one</i> pad placed (e.g. a symbol you forgot to update footprints for) – KiCad cannot route a net that only touches one physical pad.
Shield doesn’t sit flush, or some pins don’t line up, once populated	DRC and ERC only check <i>your</i> design’s internal consistency – neither one knows what a real Uno looks like. This almost always means the board outline or header positions were measured/rounded by eye instead of copied exactly from an authoritative Arduino Uno shield footprint or dimensional drawing (see the “famous gap” warning in the Shield Header Layout box).
PCBWay’s online Gerber viewer shows a blank or missing layer	Re-check your Plot dialog’s layer selection before exporting – a common miss is forgetting to include both silkscreen layers or the board-outline (Edge.Cuts) layer, which the fab needs to know your board’s physical shape.